

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I

—o0o—



BÀI TẬP LỚN

Đề bài: “THUẬT TOÁN A*”

Giảng Viên Bộ môn: TS. Nguyễn Kiều Linh

Hà Nội, 05/2024

Mục lục

Danh sách thành viên	ii
Giới thiệu thuật toán	iii
Trình bày thuật toán	vii
Demo thuật toán	xv
Nhận xét	xvi

Danh sách thành viên

Đinh Hoàng Danh - B23DCCN120

Nguyễn Thành Hưng - B23DCCN372

Nguyễn Khắc Thành Đạt - B23DCCN134

Đào Đức Đông - B23DCCN162

Lê Duy Hoàng - B23DCCN330

Trần Hưng Trung - B23DCKH123

Nguyễn Danh Thái - B23DCKH105

Giang Thủy Tiên - B23DCCN814

Giới thiệu thuật toán

I. Tổng quan về thuật toán A*

Thuật toán A* là một trong những thuật toán tìm đường đi ngắn nhất nổi bật và được sử dụng rộng rãi nhất hiện nay trong lĩnh vực khoa học máy tính và trí tuệ nhân tạo. A* được phát triển vào năm **1968** bởi **Peter Hart**, **Nils Nilsson** và **Bertram Raphael** trong bối cảnh nghiên cứu robot tự động tại SRI International. Thuật toán ra đời như một sự kết hợp giữa hai hướng tiếp cận trước đó là **Dijkstra** (mở rộng theo chi phí thấp nhất) và **Best-First Search** (ưu tiên theo độ gần đích), nhằm cân bằng giữa tính **chính xác** và **hiệu quả tìm kiếm**.

II. Nguyên lý hoạt động

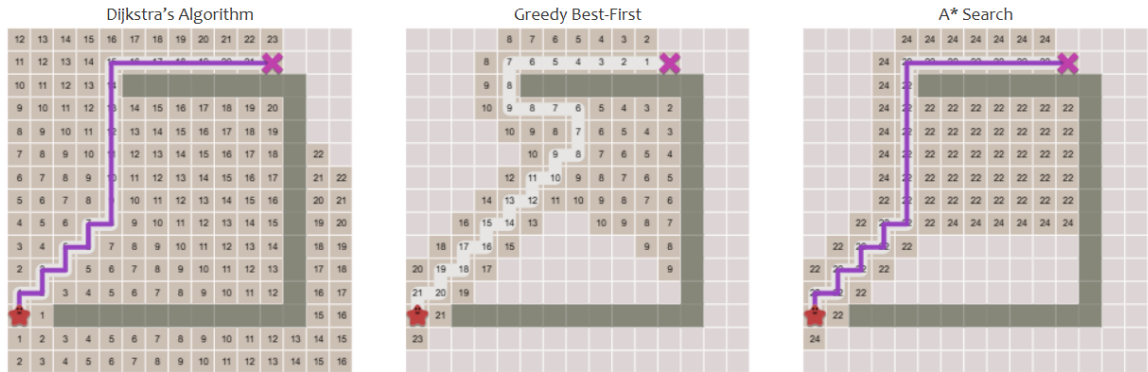
Mục tiêu của A* là tìm đường đi ngắn nhất từ một điểm xuất phát đến một điểm đích trên đồ thị, có thể là bản đồ lưới, cây trạng thái, hay mạng lưới giao thông. Điểm đặc biệt giúp A* nổi bật là việc sử dụng một **hàm đánh giá tổng hợp** $f(n)$:

$$f(n) = g(n) + h(n) \tag{1}$$

Trong đó:

- $g(n)$ là chi phí thực tế để đi từ nút bắt đầu đến nút n
- $h(n)$ là hàm heuristic ước tính chi phí từ nút n đến nút đích
- $f(n)$ là tổng chi phí dự kiến của đường đi đi qua nút n

Việc kết hợp này cho phép A* **hướng sự tìm kiếm về phía mục tiêu**, thay vì mở rộng đều theo mọi hướng như BFS, đồng thời vẫn đảm bảo tính **tối ưu**, như thuật toán Dijkstra. Nếu $h(n)$ là một hàm **admissible** (không bao giờ đánh giá quá thấp chi phí thật), A* sẽ luôn tìm được đường đi ngắn nhất nếu nó tồn tại.



Hình 1: Minh họa các thuật toán tìm đường đi

III. Ưu điểm nổi bật của thuật toán A*

- **Đầy đủ (Completeness):** Nếu có đường đi đến đích, A* chắc chắn sẽ tìm được.
- **Tối ưu (Optimality):** Nếu $h(n)$ được thiết kế tốt, thuật toán sẽ tìm ra đường đi ngắn nhất.
- **Hiệu quả (Efficiency):** So với thuật toán tìm kiếm mù, A* thường mở rộng ít nút hơn.

IV. Ứng dụng thực tế

Thuật toán A* được sử dụng trong nhiều lĩnh vực khác nhau, đặc biệt là trong các bài toán liên quan đến **tìm kiếm đường đi và tối ưu hóa di chuyển**:

- **Hệ thống bản đồ và định tuyến (Routing):** A* là nền tảng trong các công cụ tìm đường như **Google Maps, OpenStreetMap** (qua các engine

như GraphHopper, OSRM) và **Mapbox**. Thuật toán giúp xác định quãng đường ngắn nhất, thời gian di chuyển tối ưu, hoặc lộ trình tránh kẹt xe.

- **Trí tuệ nhân tạo trong trò chơi (Game AI):** A* được ứng dụng phổ biến trong các game chiến thuật và game nhập vai như **Warcraft III**, để định hướng di chuyển cho nhân vật hoặc đơn vị trong môi trường có chướng ngại vật. Khi người chơi ra lệnh di chuyển, thuật toán sẽ tự động tìm đường phù hợp và tránh va chạm.



- **Robot tự hành và lập kế hoạch di chuyển (Path Planning):** Trong lĩnh vực robotics, A* giúp các robot (ví dụ: robot giao hàng, robot hút bụi, xe tự lái) lên kế hoạch di chuyển từ vị trí hiện tại đến mục tiêu, vừa đảm bảo tối ưu hóa quãng đường, vừa tránh được các vật thể cản trở.



- **Xử lý ngôn ngữ tự nhiên (Natural Language Processing):** A* được sử dụng trong các bài toán như **phân tích cú pháp (parsing)**, nơi việc tìm ra cấu trúc câu đúng nhất tương ứng với chi phí thấp nhất có thể được diễn đạt như một bài toán tìm đường trong cây trạng thái ngôn ngữ.
- **Tối ưu hóa hệ thống và quản lý tài nguyên:** Trong một số hệ điều hành hoặc công cụ quản lý bộ nhớ, A* còn được ứng dụng vào các tác vụ như **garbage collection** (thu gom bộ nhớ không dùng đến), tìm trình tự tối ưu để giải phóng bộ nhớ.

Trình bày thuật toán

I. Hàm đánh giá $f(\mathbf{x})$

A^* lưu giữ một tập các lời giải chưa hoàn chỉnh, nghĩa là các đường đi qua đồ thị, bắt đầu từ nút xuất phát. Tập lời giải này được lưu trong một **hàng đợi ưu tiên** (priority queue). Thứ tự ưu tiên gán cho một đường đi $\mathbf{x}$ được quyết định bởi hàm:

$$f(\mathbf{x}) = g(\mathbf{x}) + h(\mathbf{x}) \quad (2)$$

Trong đó:

- $g(\mathbf{x})$: chi phí thực sự từ điểm xuất phát đến $\mathbf{x}$.
- $h(\mathbf{x})$: ước lượng chi phí nhỏ nhất từ $\mathbf{x}$ đến đích (heuristic).

Lưu ý: A^* đảm bảo tìm được lời giải tối ưu nếu hàm $h(\mathbf{x})$ là *admissible*, tức không đánh giá quá thấp chi phí thực sự (nghĩa là: $h(\mathbf{x}) \leq h^*(\mathbf{x})$ với $h^*(\mathbf{x})$ là chi phí thật sự nhỏ nhất từ $\mathbf{x}$ đến goal).

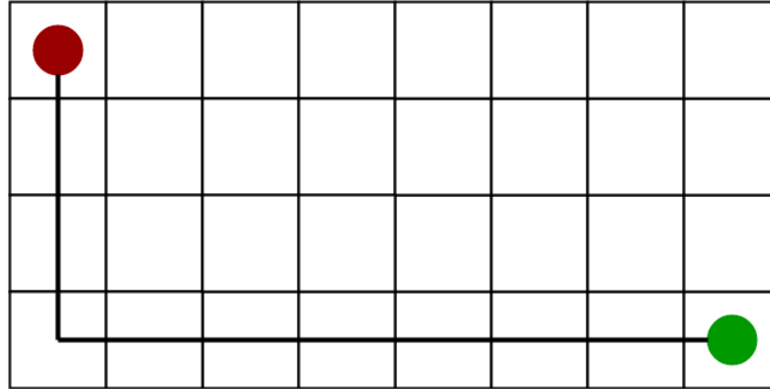
Các cách tính hàm Heuristic

1. Khoảng cách Manhattan (Manhattan Distance)

Áp dụng cho lưới ô vuông, chỉ cho phép di chuyển theo 4 hướng (lên, xuống, trái, phải).

$$h(n) = |x_1 - x_2| + |y_1 - y_2| \quad (3)$$

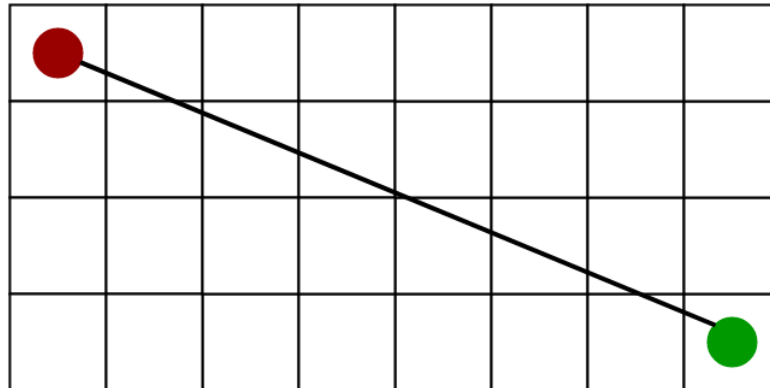
Trong đó (x_1, y_1) là tọa độ của nút hiện tại và (x_2, y_2) là tọa độ của nút đích.



Hình 2: Minh họa khoảng cách Manhattan (giả sử điểm màu đỏ là ô nguồn và điểm màu xanh lá là ô đích)

2. Khoảng cách Euclidean (Euclidean Distance)

Áp dụng cho không gian 2D hoặc 3D, cho phép di chuyển theo mọi hướng.



Hình 3: Minh họa khoảng cách Euclidean (giả sử điểm màu đỏ là ô nguồn và điểm màu xanh lá là ô đích)

Trong không gian 2D:

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad (4)$$

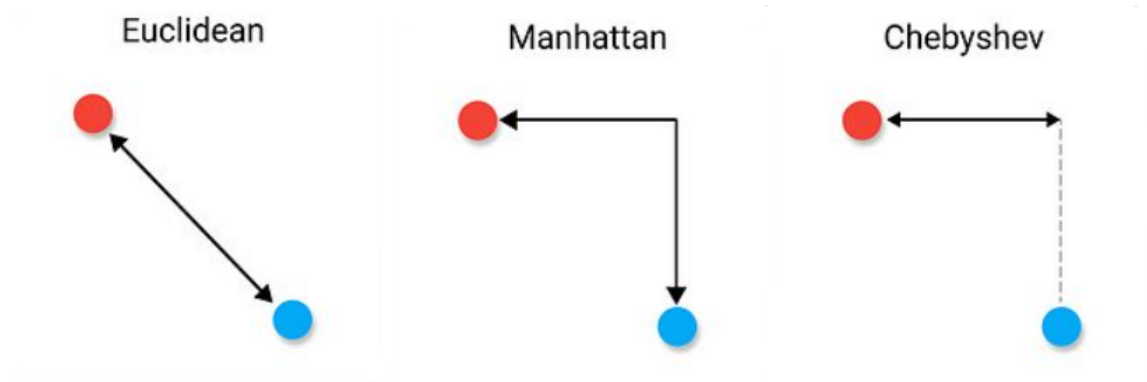
Trong không gian 3D:

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \quad (5)$$

3. Khoảng cách Chebyshev (Chebyshev Distance)

Áp dụng khi tất cả các bước di chuyển (bao gồm cả đường chéo) có chi phí bằng nhau.

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|) \quad (6)$$



Hình 4: Minh họa các loại khoảng cách heuristic

4. Hàm heuristic tùy chỉnh (Custom Heuristic)

Đối với các vấn đề phức tạp như tìm đường trong bản đồ thực tế, trò chơi, robot... ta có thể thiết kế hàm heuristic riêng phù hợp với đặc thù bài toán.

Một hàm heuristic $h(n)$ được coi là tốt khi:

1. **Admissible:** $h(n)$ không bao giờ ước lượng quá chi phí thực tế ($h(n) \leq h^*(n)$)
2. **Consistent:** $h(n) \leq c(n, a, n') + h(n')$ với mọi n' là nút kề của n
3. **Informative:** $h(n)$ nên càng gần với chi phí thực tế càng tốt
4. **Hiệu quả tính toán:** $h(n)$ phải dễ tính toán

Việc lựa chọn hàm heuristic phù hợp là yếu tố quyết định hiệu suất của thuật toán A^* .

Để tối ưu hiệu năng, hàng đợi ưu tiên thường được triển khai bằng **cây heap nhị phân tối thiểu** (min-heap), hoặc **hàng đợi ưu tiên dạng Fibonacci heap** nếu cần thao tác decrease-key hiệu quả hơn.

II. Giải thuật

Thuật toán A*

```
1: function A*(start_point, goal)
2:   var closed := empty set ▷ Tập hợp các nút đã được xét để tránh duyệt lại (tập
   đóng)
3:   var q := create_queue(create_path(start_point))    ▷ Hàng đợi ưu tiên chứa
   các đường đi chưa hoàn chỉnh
4:   while q is not empty do
5:     var p := get_first_element(q) ▷ Lấy ra đường đi p có giá trị f(p) nhỏ nhất
6:     var x := last node of p        ▷ x là nút cuối cùng trong đường đi p
7:     if x in closed then
8:       continue    ▷ Nếu nút x đã được xử lý trước đó, bỏ qua để tránh lặp
9:     end if
10:    if x = goal then
11:      return p      ▷ Nếu x là đích, trả về đường đi hiện tại là lời giải
12:    end if
13:    add x to closed  ▷ Đánh dấu x là đã xử lý (thêm vào tập đóng)
14:    for all y in next_paths(p) do
15:      add_to_queue(q, y)
                        ▷ Thêm các đường đi mới vào hàng đợi, sắp theo  $f = g + h$ 
16:    end for
17:  end while
18:  return failure    ▷ Nếu không còn đường đi nào để xét và chưa đến đích
19: end function
```

Các thành phần trong thuật toán

Thành phần	Ý nghĩa
closed	Tập các nút đã mở rộng, giúp tránh xét lại các nút đã duyệt để không đi vào chu trình.
q	Hàng đợi ưu tiên chứa các đường đi tạm thời (mỗi đường đi là 1 danh sách các nút), sắp xếp theo $f(x) = g(x) + h(x)$.
get_first_element(q)	Lấy ra đường đi có giá trị f thấp nhất từ hàng đợi.
next_paths(p)	Hàm tạo ra các đường đi mới bằng cách mở rộng đường đi p sang các nút kề chưa được duyệt.
add_to_queue(q, y)	Thêm đường đi mới vào hàng đợi ưu tiên để xét sau.
$g(x)$	Chi phí thực sự từ điểm bắt đầu đến nút x (tổng trọng số các cạnh đã đi qua trong đường đi).
$h(x)$	Heuristic ước lượng chi phí còn lại từ x đến đích.
$f(x)$	Tổng chi phí ước lượng của đường đi qua x đến goal: $f(x) = g(x) + h(x)$. Thấp hơn thì ưu tiên xét trước.

Bảng 1: Bảng mô tả các thành phần trong thuật toán A^*

III. Cơ chế hoạt động và các khái niệm liên quan trong A^*

Trong đó, $next_paths(p)$ trả về tập hợp các đường đi tạo bởi việc kéo dài p thêm một nút kề cạnh. Giả thiết rằng hàng đợi được sắp xếp tự động bởi giá trị của hàm f .

- **Closed set** (*tập đóng*): lưu giữ tất cả các nút cuối cùng của đường đi p , tức các nút mà từ đó các đường đi mới đã được mở rộng.
- Việc sử dụng **closed set** giúp tránh lặp lại các chu trình, từ đó tạo ra thuật toán graph search (*tìm kiếm theo đồ thị*).
- **Queue** (*hàng đợi*): là cấu trúc dữ liệu lưu trữ các đường đi đang được xem xét; đôi khi được gọi là **open set** (*tập mở*).
- Nếu không sử dụng **closed set**, ta thu được thuật toán **tree search** (*tìm kiếm theo cây*), với điều kiện:

- Có thể đảm bảo tồn tại ít nhất một lời giải, hoặc
- Hàm **next_paths(p)** (*các đường đi tiếp theo (p)*) được chỉnh sửa để loại bỏ các chu trình.

IV. Độ phức tạp của thuật toán A* và hiệu suất thuật toán A*

Độ phức tạp thời gian

- Phụ thuộc vào **chất lượng của hàm heuristic**.
- Trong **trường hợp xấu nhất**: số nút mở rộng **tăng theo hàm mũ** so với độ dài lời giải.
- Nếu **sai số của h(x)** so với heuristic tối ưu $h^*(x)$ thỏa mãn:

$$|h(x) - h^*(x)| \leq O(\log h^*(x)) \quad (7)$$

thì độ phức tạp **có thể giảm xuống mức đa thức**.

Độ phức tạp bộ nhớ

- A* có thể cần lưu trữ **một lượng lớn nút**, tăng **theo hàm mũ** trong trường hợp xấu.

Giải pháp cải tiến bộ nhớ

- **IDA*** (Iterative Deepening A*): Lặp sâu dần theo ngưỡng f.
- **MA*** (Memory-Bounded A*): A* với bộ nhớ giới hạn.
- **SMA*** (Simplified Memory-Bounded A*): Phiên bản đơn giản hơn của MA*.

V. So sánh với các thuật toán khác

Một số thuật toán tìm kiếm đường đi kinh điển có thể xem là trường hợp đặc biệt của A^* :

Tìm kiếm theo chiều rộng (BFS)

- Dừng khi mọi bước đi đều có chi phí như nhau (ví dụ: mỗi bước đi tốn 1 đơn vị).
- Tương đương với A^* khi: $g(n)$ = số bước đã đi, còn $h(n) = 0$ (không dùng heuristic).
- BFS mở rộng đều theo từng “lớp” từ điểm bắt đầu.
- Ưu điểm: Tìm được đường ngắn nhất trên đồ thị không trọng số.
- Nhược điểm: Không hiệu quả khi đồ thị lớn hoặc có trọng số khác nhau, vì không có hướng dẫn cụ thể đến đích.

Thuật toán Dijkstra

- Tương đương với A^* khi $h(n) = 0$ cho mọi nút.
- Dijkstra luôn mở rộng nút có chi phí $g(n)$ nhỏ nhất.
- Ưu điểm: Đảm bảo tìm được đường đi tối ưu trên đồ thị có trọng số.
- Nhược điểm: Vì không có định hướng cụ thể về phía đích, thuật toán có thể mở rộng rất nhiều nút không cần thiết.
- A^* cải tiến bằng cách thêm $h(n)$ để tập trung hơn vào hướng đi đến đích, giảm số nút phải xét.

Thuật toán tham lam (Greedy Best-First Search)

- Tương đương với A^* khi bỏ qua $g(n)$, chỉ dùng $h(n)$, tức là $f(n) \approx h(n)$.
- Luôn chọn nút có ước lượng gần đích nhất để mở rộng.

- Ưu điểm: Rất nhanh, vì chỉ chú ý đến việc tiến gần đến mục tiêu.
- Nhược điểm: Không đảm bảo tìm được đường tối ưu, dễ rơi vào ngõ cụt vì không tính chi phí đã đi.

Tóm lại:

A^* là sự kết hợp giữa tính chính xác của Dijkstra (nhờ $g(n)$) và khả năng định hướng nhanh của heuristic (nhờ $h(n)$).

Do đó, A^* thường cho kết quả tối ưu với hiệu suất cao hơn trong thực tế.

Demo thuật toán



Hình 5: Giao diện demo thuật toán A* với các nút bắt đầu (màu xanh lá), kết thúc (màu đỏ), các ô đã xét (màu xám), đường đi tìm được (màu vàng)

Demo A* Pathfinding

Nhấp vào liên kết dưới đây để mở demo trong trình duyệt:

[Mở Demo A* Pathfinding](#)

Nhận xét

Ưu điểm của thuật toán A^*

- **Hoàn thiện:** A^* sẽ luôn tìm thấy lời giải nếu tồn tại ít nhất một đường đi đến đích (giả sử không gian trạng thái là hữu hạn hoặc có ràng buộc chi phí tăng dương). Tính chất này giống như ở BFS, đảm bảo không bỏ sót kết quả.
- **Tối ưu:** Nếu hàm heuristic được chọn admissible – tức không bao giờ vượt quá chi phí thực tế từ một nút tới đích – thì thuật toán A^* đảm bảo tìm được đường đi có chi phí nhỏ nhất. Đặc biệt, nếu heuristic còn thỏa mãn tính nhất quán (consistency), A^* sẽ không cần mở rộng lại nút nào lần thứ hai, giúp tiết kiệm thời gian và đảm bảo tối ưu mà không lặp.
- **Hiệu quả:** A^* kết hợp cả chi phí thực tế từ điểm bắt đầu ($g(n)$) và ước lượng đến đích ($h(n)$), giúp giảm số lượng nút cần duyệt. Thực tế cho thấy với heuristic tốt, A^* mở rộng ít nút hơn rất nhiều so với việc duyệt mù như BFS hoặc Dijkstra. Nhờ ưu điểm này, A^* thường giải quyết được các bài toán tìm đường phức tạp một cách khả thi.
- **Linh hoạt và phổ dụng:** A^* có thể áp dụng cho nhiều loại bài toán khác nhau chỉ bằng cách thay đổi hàm heuristic phù hợp. Từ đường đi trên lưới ô vuông, bản đồ giao thông, trò chơi (puzzle), hay phân tích cú pháp ngôn ngữ, A^* đều tỏ ra hữu hiệu.
- **Phù hợp với nhiều bài toán thực tế:** Rất hiệu quả trong môi trường có bản đồ rõ ràng như điều hướng GPS, trí tuệ nhân tạo trong game, robot tự hành, giải bài toán tổ hợp.

Nhược điểm của thuật toán A*

- **Tiêu tốn nhiều bộ nhớ:** Do lưu trữ tất cả các nút đã duyệt (open và closed list), dung lượng bộ nhớ sử dụng tăng nhanh theo kích thước không gian trạng thái. Trong trường hợp xấu, số nút lưu trữ có thể tăng theo cấp số nhân với độ sâu lời giải và bậc phân nhánh – phức tạp không gian $O(b^d)$. Các biến thể như A* memory-bounded (SMA*), Iterative Deepening A* (IDA*) được phát triển nhằm khắc phục nhược điểm này.
- **Hiệu suất phụ thuộc vào chất lượng của hàm heuristic:** Nếu heuristic không tốt hoặc kém hiệu quả, A* sẽ hành xử gần như thuật toán Dijkstra, phải mở rộng rất nhiều nút nên thời gian chạy tăng mạnh. Hiệu quả phụ thuộc nhiều vào việc thiết kế hàm heuristic. Tìm ra một heuristic vừa admissible, vừa đủ "nhảy" là một thách thức lớn.
- **Khó áp dụng cho môi trường động:** Trong môi trường thay đổi liên tục, A* cần được chạy lại nhiều lần hoặc sử dụng biến thể như D*.
- **Cài đặt phức tạp:** Do cần tổ chức dữ liệu (hàng đợi ưu tiên, bảng đánh dấu...) và tính toán heuristic hợp lý.
- **Tính cố định của mục tiêu:** A* thiết kế cho bài toán đường đi với một mục tiêu cụ thể. Nếu cần tìm đường từ một nguồn đến nhiều đích khác nhau, phải chạy lại A* cho mỗi mục tiêu, chi phí sẽ cao hơn so với Dijkstra.
- **Hạn chế với một số loại đồ thị:** Không áp dụng được nếu đồ thị có cạnh trọng số âm. Nếu bài toán yêu cầu nhiều tiêu chí (đa mục tiêu) hoặc ràng buộc phức tạp, ta có thể phải điều chỉnh hàm đánh giá hoặc kết hợp với phương pháp khác thay vì dùng A* thuần túy.

Các yếu tố ảnh hưởng đến hiệu suất

- **Chất lượng của hàm heuristic:** Hàm heuristic càng gần với chi phí thực tế, A* càng hiệu quả.

- **Cấu trúc dữ liệu:** Việc sử dụng cấu trúc dữ liệu phù hợp như Heap nhị phân hoặc Fibonacci heap có thể cải thiện hiệu suất của A*.
- **Chiến lược tìm kiếm:** Các biến thể như tìm kiếm hai chiều (Bidirectional Search) hoặc phân cấp (Hierarchical Pathfinding) có thể giảm thiểu không gian tìm kiếm và tăng tốc độ tìm kiếm.

Khi nào nên (hoặc không nên) dùng A*

Trường hợp dùng tốt	Trường hợp nên cân nhắc
Có heuristic tốt	Không có heuristic hoặc heuristic kém
Môi trường tĩnh	Môi trường thay đổi liên tục
Không gian tìm kiếm vừa phải	Không gian quá lớn, giới hạn bộ nhớ

Kết luận: Thuật toán A* là công cụ mạnh mẽ và linh hoạt trong việc giải quyết các bài toán tìm đường đi ngắn nhất, đặc biệt khi kết hợp với hàm heuristic phù hợp. Tuy nhiên, người sử dụng cần lưu ý đến các yếu tố như bộ nhớ và chất lượng hàm heuristic để đảm bảo hiệu suất tối ưu. Dù có một số hạn chế, A* vẫn được đánh giá là lý tưởng trong nhiều trường hợp, đặc biệt với không gian trạng thái vừa và nhỏ, hoặc khi có heuristic tốt. Hiện nay, A* và các biến thể của nó vẫn tiếp tục là nền tảng cho nhiều hệ thống đường đi và tối ưu hóa.